Black-Box API Testing Report

QuickCart REST API

Part 3: Black-Box Testing (35 marks)

Tool Chain:	pytest + requests (Python 3)
API Base URL:	`http://localhost:8080/api/v1`
Docker Image:	`quickcart:latest`
Bugs Found:	19 confirmed bugs

Contents

1 Test Case Design **2**
1.1 Authentication & Header Validation (`test_auth.py`) 2
1.2 Admin / Data Inspection Endpoints (`test_admin.py`) 2
1.3 Profile (`test_profile.py`) 3
1.4 Addresses (`test_addresses.py`) 3
1.5 Products (`test_products.py`) 4
1.6 Cart (`test_cart.py`) 4
1.7 Coupons (`test_coupons.py`) 5
1.8 Checkout (`test_checkout.py`) 5
1.9 Wallet (`test_wallet.py`) 6
1.10 Loyalty Points & Orders (`test_loyalty_orders.py`) 6
1.11 Reviews & Support Tickets (`test_reviews_tickets.py`) 7

2 Bug Reports **8**

Test Case Design

All test cases below are based on the QuickCart API documentation. Tests are implemented using `pytest` and `requests`, located in `blackbox/tests/`. Each test validates correct HTTP status codes, proper JSON response structures, and correctness of returned data.

Authentication & Header Validation (`test_auth.py`)

Test	Input	Expected	Justification
`test_missing_roll_number`	GET /profile without X-Roll-Number	401	Doc states missing roll number returns 401
`test_invalid_roll_number`	X-Roll-Number: "abc"	400	Non-integer roll number returns 400
`test_missing_user_id`	GET /profile without X-User-ID	400	User-scoped endpoints require X-User-ID
`test_invalid_user_id`	X-User-ID: "abc"	400	Invalid (non-integer) user ID must fail
`test_symbol_roll_number`	X-Roll-Number: "12@34"	400	Symbols are not valid integers
`test_nonexistent_user_id`	X-User-ID: "999999"	400	Non-existent user ID must be rejected

Admin / Data Inspection Endpoints (`test_admin.py`)

Test	Input	Expected	Justification
`test_admin_get_all_users`	GET /admin/users	200	Admin can see all users
`test_admin_get_specific_user`	GET /admin/users/1	200	Fetching a known user works
`test_admin_get_nonexistent_user`	GET /admin/users/999999	404	Non-existent user must return 404
`test_admin_get_all_carts`	GET /admin/carts	200	Returns all carts across users
`test_admin_get_all_orders`	GET /admin/orders	200	Returns all orders across users
`test_admin_get_all_products`	GET /admin/products	200	Returns ALL products including inactive
`test_admin_get_all_coupons`	GET /admin/coupons	200	Returns all coupons including expired
`test_admin_get_all_tickets`	GET /admin/tickets	200	Returns all support tickets
`test_admin_get_all_addresses`	GET /admin/addresses	200	Returns all addresses across users

Profile (`test_profile.py`)

Test	Input	Expected	Justification
`test_get_profile`	GET /profile	200 + name, phone	Basic valid retrieval
`test_update_profile_valid`	PUT name="Test User", phone="1234567890"	200	Normal valid update
`test_update_profile_boundary_valid`	name=2 chars, name=50 chars	200	Boundary min/max for name length
`test_update_profile_invalid_name_length`	name=1 char, name=51 chars	400	Below/above allowed name range
`test_update_profile_invalid_phone_format`	phone=9 digits, 11 digits, "123abc4567"	400	Phone must be exactly 10 digits
`test_update_profile_wrong_type`	name=12345 (int)	400	Wrong data type validation

Addresses (`test_addresses.py`)

Test	Input	Expected	Justification
`test_add_address_valid_labels`	label=HOME/OFFICE/OTHER	200 + new address	All three valid labels accepted
`test_add_address_invalid_label`	label="INVALID"	400	Only HOME/OFFICE/OTHER allowed
`test_add_address_invalid_street_length`	street=4 chars, street=101 chars	400	Street must be 5–100 chars
`test_add_address_invalid_pincode`	pincode=5 digits, 7 digits, "abcdef"	400	Pincode must be exactly 6 digits
`test_add_address_invalid_city_length`	city=1 char, city=51 chars	400	City must be 2–50 chars
`test_default_address_logic`	Add two addresses, set second as default	First=False, Second=True	Only one default at a time
`test_update_address_street`	PUT new street	200 + new street	Update must return new data
`test_update_address_ignored_fields`	PUT label/city/pincode	Original unchanged	Only street and is_default changeable
`test_delete_non_existent_address`	DELETE /addresses/99999	404	Non-existent address returns 404
`test_add_address_response_includes_all_fields`	POST valid address	All 6 fields present	Complete JSON structure

Test	Input	Expected	Justification
`test_delete_existing_address`	DELETE real address	200/204	Verifies successful deletion

Products (`test_products.py`)

Test	Input	Expected	Justification
`test_get_products_active_only`	GET /products	All is_active=True	Inactive products must be hidden
`test_get_single_product_valid`	GET /products/250	200 + all fields	Basic valid retrieval
`test_get_single_product_not_found`	GET /products/999999	404	Non-existent product returns 404
`test_products_filter_category`	GET /products?category=Fruits	All category=Fruits	Category filter works
`test_products_search_name`	GET /products?search=Apple	200	Name search returns results
`test_products_sort_price`	sort=price_asc, price_desc	Prices sorted correctly	Sort works both ways
`test_products_price_matches_admin`	Compare user prices to admin prices	Prices must match	Price must be exact DB price

Cart (`test_cart.py`)

Test	Input	Expected	Justification
`test_cart_add_valid`	product_id=250, qty=2	200	Normal add-to-cart
`test_cart_add_invalid_quantity_zero`	qty=0	400	Quantity must be at least 1
`test_cart_add_invalid_quantity_negative`	qty=-1	400	Negative quantities invalid
`test_cart_add_wrong_data_type`	qty="two"	400	Wrong data type for quantity
`test_cart_add_missing_product_id`	No product_id	400	Missing required field
`test_cart_add_stock_exceeded`	qty=9999	400	Cannot exceed stock
`test_cart_add_non_existent`	product_id=99999	404	Non-existent product
`test_cart_duplicate_add_accumulates`	Add qty=1, then qty=2	Total qty=3	Quantities accumulate, not replace
`test_cart_update_quantity`	Update to qty=5, then qty=0	200 then 400	Valid update; zero rejected

Test	Input	Expected	Justification
`test_cart_remove`	Remove existing, then non-existent	200 then 404	Remove works; non-existent 404
`test_cart_totals_calculation`	Add 2 products, check total	total = sum of subtotals	Total calculation correctness
`test_cart_clear`	Add item then clear	Cart has 0 items	Clear empties the cart
`test_cart_item_subtotal_per_item`	Add qty=3	subtotal = qty × unit_price	Per-item subtotal verification

Coupons (`test_coupons.py`)

Test	Input	Expected	Justification
`test_coupon_apply_valid`	Active coupon, low min_cart_value	200	Valid coupon accepted
`test_coupon_expired`	Expired/non-existent coupon	400/404	Expired coupons rejected
`test_coupon_minimum_cart_value`	Coupon min > cart total	400	Cart must meet minimum value
`test_coupon_discount_calculation`	Apply PERCENT coupon	discount = subtotal×pct/100, capped	Discount calculated correctly
`test_coupon_remove`	Remove applied coupon	200	Coupon removal works

Checkout (`test_checkout.py`)

Test	Input	Expected	Justification
`test_checkout_empty_cart`	Checkout with empty cart	400	Cannot checkout empty cart
`test_checkout_invalid_method`	payment_method="CASH"	400	Only COD/WALLET/CARD allowed
`test_checkout_missing_payment_method`	Empty JSON body	400	Missing required field
`test_checkout_cod_limit`	COD with total > 5000	400	COD not allowed > 5000
`test_checkout_valid_cod`	COD with low total	200, status=PENDING	COD starts as PENDING
`test_checkout_valid_wallet`	WALLET payment	200, status=PENDING	WALLET starts as PENDING
`test_checkout_valid_card`	CARD payment	200, status=PAID	CARD starts as PAID

Test	Input	Expected	Justification
`test_checkout_gst_and_stock_reduction`	CARD checkout	GST=5% of subtotal, stock reduces	Validates GST and stock management

Wallet (`test_wallet.py`)

Test	Input	Expected	Justification
`test_wallet_get`	GET /wallet	200 + wallet_balance	Basic retrieval
`test_wallet_add_boundary_100000`	amount=100000	200	Boundary max: exactly 100000 accepted
`test_wallet_add_valid_and_verify`	Add 50, check balance	after = before + 50	Verifies exact addition
`test_wallet_add_invalid_amount`	amount=100001, amount=0	400	Over limit and zero rejected
`test_wallet_add_wrong_type`	amount="hundred"	400	Wrong data type
`test_wallet_add_negative_amount`	amount=-50	400	Negative amounts rejected
`test_wallet_pay_exact_deduction`	Pay 100	after = before − 100	No extra amount deducted
`test_wallet_pay_insufficient`	Pay amount > balance	400	Insufficient funds rejected
`test_wallet_pay_invalid_amount`	amount=0, amount=-100	400	Zero and negative pay rejected

Loyalty Points & Orders (`test_loyalty_orders.py`)

Test	Input	Expected	Justification
`test_loyalty_get`	GET /loyalty	200 + loyalty_points	Basic retrieval
`test_loyalty_redeem`	Redeem 0, -1, valid, over-balance	400/200/400	All redeem edge cases
`test_orders_get_all`	GET /orders	200	Basic order listing
`test_orders_get_single_and_non_existent`	GET existing, GET /orders/99999	200 then 404	Valid retrieval and 404 for missing
`test_orders_invoice`	GET /orders/{id}/invoice	Has subtotal, gst_amount, total	Invoice structure validation

Test	Input	Expected	Justification
`test_orders_cancel_restores_stock`	Cancel order, check stock	Stock restored	Cancellation must restore inventory
`test_orders_cancel_non_existent`	POST /orders/99999/cancel	404	Non-existent order can't be cancelled
`test_orders_cancel_delivered`	Cancel DELIVERED order	400	Delivered orders cannot be cancelled

Reviews & Support Tickets (`test_reviews_tickets.py`)

Test	Input	Expected	Justification
`test_review_get_and_average`	POST rating=3, rating=4, check avg	avg is float (3.5)	Average must be decimal
`test_review_valid_and_boundaries`	rating=1, rating=5	200	Boundary min/max for rating
`test_review_invalid_rating`	rating=0, rating=6	400	Outside 1–5 range rejected
`test_review_comment_boundaries`	comment="" (empty), comment=201 chars	400	Comment must be 1–200 chars
`test_review_wrong_data_type`	rating="five"	400	Wrong data type validation
`test_review_missing_fields`	Missing comment, missing rating	400	Required fields must be present
`test_support_ticket_get_all`	GET /support/tickets	200	Basic retrieval
`test_support_ticket_missing_fields`	Missing message, missing subject	400	Required fields must be present
`test_support_ticket_create_valid`	subject=valid, message=valid	200, status=OPEN	New ticket starts OPEN
`test_support_ticket_create_boundaries`	subject=4 chars / 101 chars, msg=501 chars	400	Subject 5–100, message 1–500
`test_support_ticket_valid_transitions`	OPEN→IN_PROGRESS→CLOSED	200 each step	Valid forward transitions
`test_support_ticket_closed_to_open_invalid`	CLOSED→OPEN	400	Reverse transitions not allowed
`test_support_ticket_invalid_transition`	OPEN→CLOSED (skip)	400	Must go through IN_PROGRESS
`test_support_ticket_message_saved_exactly`	POST with specific message text	Message matches exactly	Full message saved exactly

Bug Reports

Bug 1: Address Update Returns Old Data

Endpoint: `PUT /api/v1/addresses/{address_id}`
Method: PUT **URL:** `http://localhost:8080/api/v1/addresses/<id>`
Headers: `{"X-Roll-Number": "2024101006", "X-User-ID": "1"}`
Body: `{"street": "New Street Name"}`

Expected: API returns address with `"street": "New Street Name"`.
Actual: API returned the old address data (`"street": "Valid St"`), ignoring the update rule.

Bug 2: Cart Allows Zero Quantity

Endpoint: `POST /api/v1/cart/add`
Body: `{"product_id": 250, "quantity": 0}`

Expected: `400 Bad Request` — quantity must be at least 1.
Actual: API returned `200 OK` and added an invalid quantity to the cart.

Bug 3: Cart Total and Subtotal Calculation is Incorrect

Endpoint: `GET /api/v1/cart`

Expected: Adding 5× "Apple – Red" (price 120 each) ⇒ subtotal=600, total=600.
Actual: API returned `subtotal: 88` and `total: 0`. Adding 2× gives subtotal=-16. Fundamental calculation logic error.

Bug 4: Checkout Allows Empty Cart

Endpoint: `POST /api/v1/checkout`
Body: `{"payment_method": "COD"}`

Expected: `400 Bad Request` — cart is empty.
Actual: API returned `200 OK` and processed a checkout for 0 items.

Bug 5: Support Ticket Allows Jumping Statuses

Endpoint: `PUT /api/v1/support/tickets/{ticket_id}`
Body: `{"status": "CLOSED"}` (on an OPEN ticket)

Expected: `400 Bad Request` — ticket must go OPEN → IN_PROGRESS → CLOSED.
Actual: API returned `200 OK` and allowed OPEN ticket to immediately become CLOSED.

Bug 6: Addresses Allow Non-digit Pincodes

Endpoint: `POST /api/v1/addresses`
Body: `{"label": "HOME", "street": "Valid St", "city": "City", "pincode": "abcdef"}`

Expected: `400 Bad Request` — pincode must be exactly 6 digits.
Actual: API returned `200 OK` and saved the invalid pincode.

Bug 7: Profile Allows Non-digit Phone Numbers

Endpoint: `PUT /api/v1/profile`
Body: `{"name": "Test User", "phone": "123abc4567"}`

Expected: `400 Bad Request` — phone must be exactly 10 digits.
Actual: API returned `200 OK` and saved the invalid phone string.

Bug 8: Review Average Rating is Integer-Divided

Endpoint: `GET /api/v1/products/{product_id}/reviews`

Expected: Proper decimal average, e.g. 3.5 for ratings of 3 and 4.
Actual: API returned `3` (integer floor division) instead of `3.5`.

Bug 9: Support Ticket Allows Reopening Closed Tickets

Endpoint: `PUT /api/v1/support/tickets/{ticket_id}`
Body: `{"status": "OPEN"}` (on a CLOSED ticket)

Expected: `400 Bad Request` — status can only move forward.
Actual: API returned `200 OK` and allowed the CLOSED→OPEN reverse transition.

Bug 10: Wallet Pay Deducts Extra Hidden Amount

Endpoint: `POST /api/v1/wallet/pay`
Body: `{"amount": 50}`

Expected: Balance decreases by exactly 50. (584.79 → 534.79)
Actual: 50.80 was deducted instead of 50. Balance became 533.99. The doc states "no extra amount is taken."

Bug 11: Standard Product API Returns Wrong Price

Endpoint: `GET /api/v1/products/250`

Expected: Price matching the exact DB value. Admin shows `284.61`.
Actual: User-facing endpoint returned `"price": 280` instead of `284.61`.

Bug 12: Coupon Discount Calculated as Zero

Endpoint: `POST /api/v1/coupon/apply` + `GET /api/v1/cart`
Body: `{"coupon_code": "WELCOME50"}`

Expected: WELCOME50 (FIXED, value=50, min=100) should show discount=50 in cart.
Actual: Coupon applied (200 OK) but `GET /cart` showed `total: 0` with no discount field. Root cause is Bug 3 (broken cart total).

Bug 13: Inactive Products Can Be Added to Cart

Endpoint: `POST /api/v1/cart/add`
Body: `{"product_id": 90, "quantity": 1}` (product 90 is `is_active: false`)

Expected: `400 Bad Request` or `404 Not Found`.
Actual: API returned `200 OK` and added the inactive product to the cart.

Bug 14: WALLET Checkout Ignores Insufficient Balance

Endpoint: `POST /api/v1/checkout`
Body: `{"payment_method": "WALLET"}` (cart total > wallet balance)

Expected: `400 Bad Request` — insufficient wallet funds.
Actual: API returned `200 OK`, placed the order as PENDING, and did not deduct any money. Balance validation is completely missing.

Bug 15: Support Ticket Accepts Invalid Status Strings

Endpoint: `PUT /api/v1/support/tickets/151`
Body: `{"status": "INVALID_STATUS"}`

Expected: `400 Bad Request` — only OPEN, IN_PROGRESS, CLOSED are valid.
Actual: API returned `200 OK` and stored `"INVALID_STATUS"` as the ticket status.

Bug 16: Duplicate Reviews Allowed on Same Product

Endpoint: `POST /api/v1/products/2/reviews`
Body: `{"rating": 3, "comment": "Second Review"}` (user already reviewed product 2)

Expected: `400 Bad Request` — user can only review a product once.
Actual: API returned `200 OK` and created a duplicate review under the same user ID and product ID.

Bug 17: Cart Update Bypasses Stock Limit Validation

Endpoint: `POST /api/v1/cart/update`
Body: `{"product_id": 250, "quantity": 99999}`

Expected: `400 Bad Request` — quantity exceeds available stock.
Actual: API returned `200 OK` and updated the cart item to quantity 99999, ignoring the stock limit.

Bug 18: Review Endpoint Allows Ghost Reviews for Non-existent Products

Endpoint: `POST /api/v1/products/999999/reviews`
Body: `{"rating": 5, "comment": "Ghost review"}`

Expected: `404 Not Found` — product 999999 does not exist.
Actual: API returned `200 OK` and created a review tied to a non-existent product ID.

Bug 19: Support Ticket Allows Reverse IN_PROGRESS to OPEN Transition

Endpoint: `PUT /api/v1/support/tickets/154`
Body: `{"status": "OPEN"}` (ticket is currently IN_PROGRESS)

Expected: `400 Bad Request` — spec says no reverse transitions allowed.
Actual: API returned `200 OK` and reverted the ticket status back to OPEN.